香港科技大學(廣州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

机器学习基础

讲师：Hao Wang,
AI Thrust，信息枢纽

鸣谢：大部分幻灯片修改自 Standford 的 CS231n

今日大纲

- 线性代数
- K 最近邻
- 支持向量机
- 梯度下降
- 计算图



标量

- **简单操作**

$$c = a + b$$

$$c = a \cdot b$$

$$c = \sin a$$

- **长度**

$$|a| = \begin{cases} a & \text{if} a > 0 \\ -a & \text{otherwise} \end{cases}$$

$$|a + b| \leq |a| + |b|$$

$$|a \cdot b| = |a| \cdot |b|$$



向量

- **简单运算**

逐元素操作

$\boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b}$ 其中 $c_i = a_i + b_i$

$\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b}$ 其中 $c_i = a_i b_i \boldsymbol{c} = \sin \boldsymbol{a}$

其中 $c_i = \sin a_i$

- **长度（范数）**

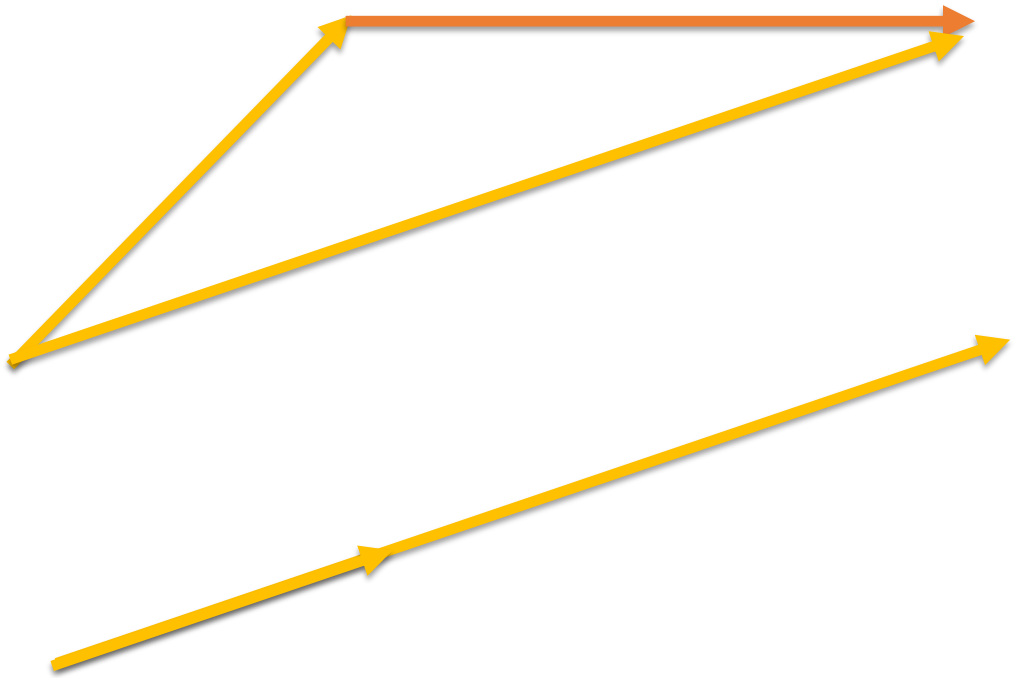
$$||\boldsymbol{x}|| = \sqrt{\sum_i x_i^2}$$

$$||\boldsymbol{a} + \boldsymbol{b}|| \leq ||\boldsymbol{a}|| + ||\boldsymbol{b}||$$

$$||a \cdot \boldsymbol{b}|| = |a| \cdot ||\boldsymbol{b}||$$



向量



$$c = a + b$$

$$c = \alpha \cdot b$$

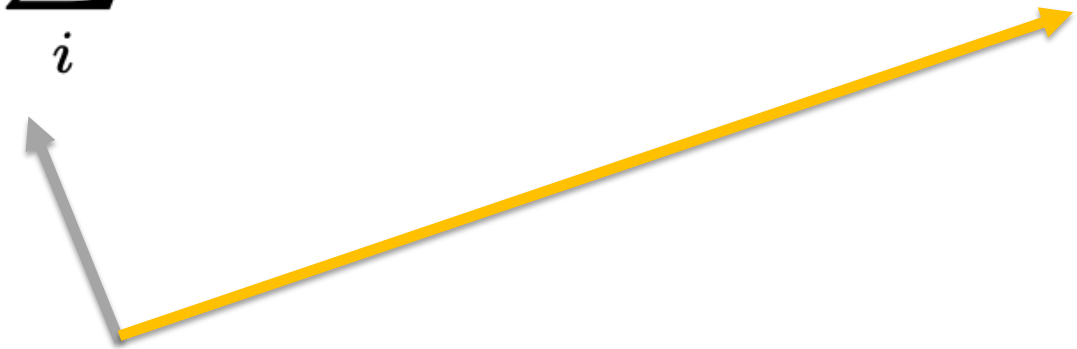
向量

- **点积**

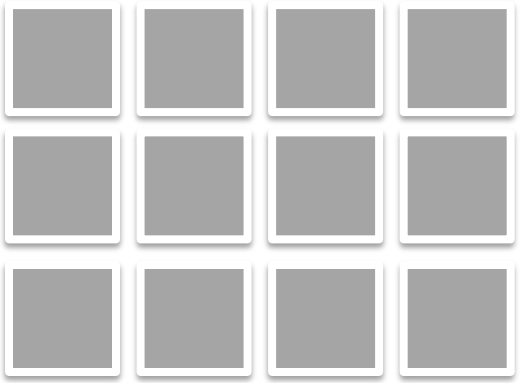
$$a^\top b = \sum_i a_i b_i$$

- **正交：点积** □□

$$a^\top b = \sum_i a_i b_i = 0$$



矩阵



- **简单运算**

$$C = A + B \qquad \text{where } C_{ij} = A_{ij} + B_{ij}$$

$$C = \alpha \cdot B \qquad \text{where } C_{ij} = \alpha B_{ij}$$

$$C = \sin A \qquad \text{where } C_{ij} = \sin A_{ij}$$



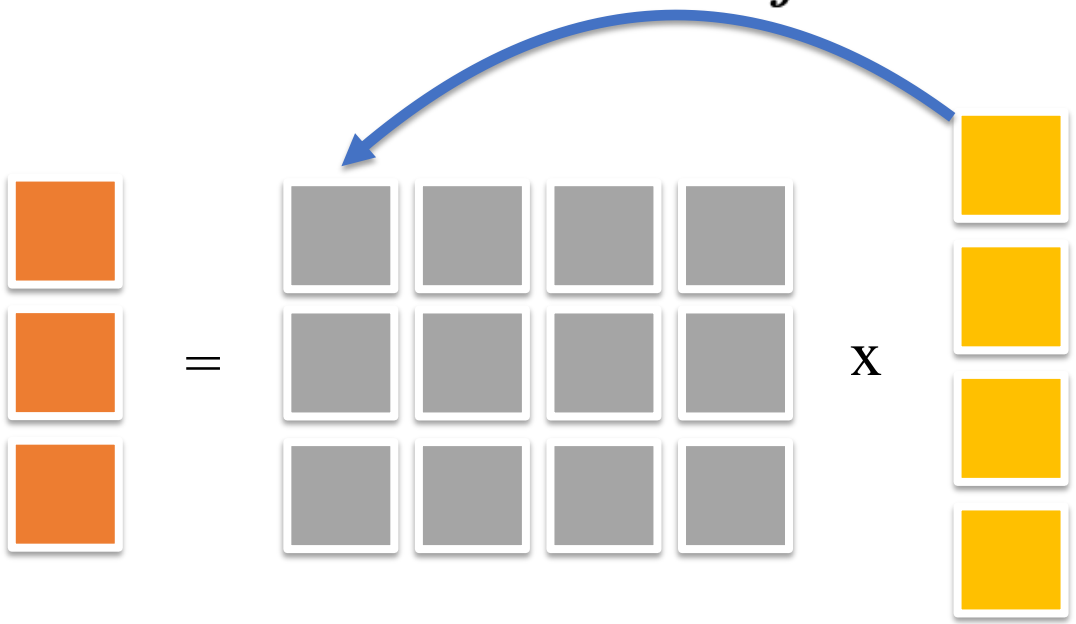
矩阵

- **乘法（矩阵向量）**

$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$

形状：Axb 矩阵乘法 b x cà A x C

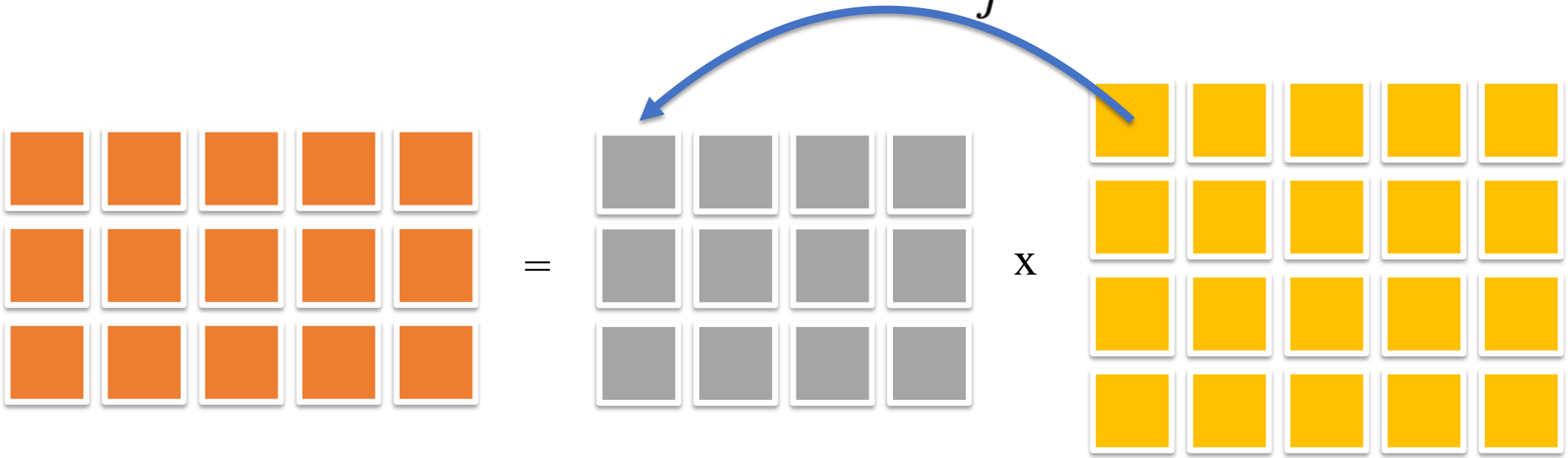
Axbxc 矩阵乘法 cxd ->Axbxd



矩阵

- **乘法（矩阵 矩阵）**

$$C = AB \text{ where } C_{ik} = \sum_j A_{ij} B_{jk}$$



矩阵

- **长度（范数）**

$$||B||_{Frob} \leq \left[\sum_{ij} B_{ij}^2 \right]^{\frac{1}{2}}$$

$$||B - L||_{Frob} \sum ||B||_{Frob} - ||L||_{Frob}$$

$$||eB||_{Frob} \leq |e| A||B||_{Frob}$$

其他范数：p-范数、核范数、谱范数

……https://en.wikipedia.org/wiki/Matrix_范数

图像分类

K 最近邻

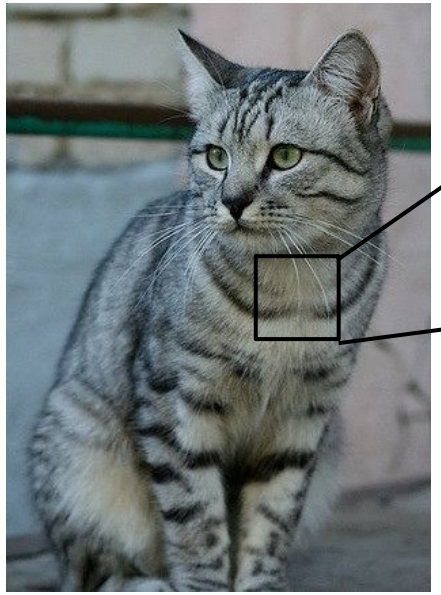


问题



Cat

问题



```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

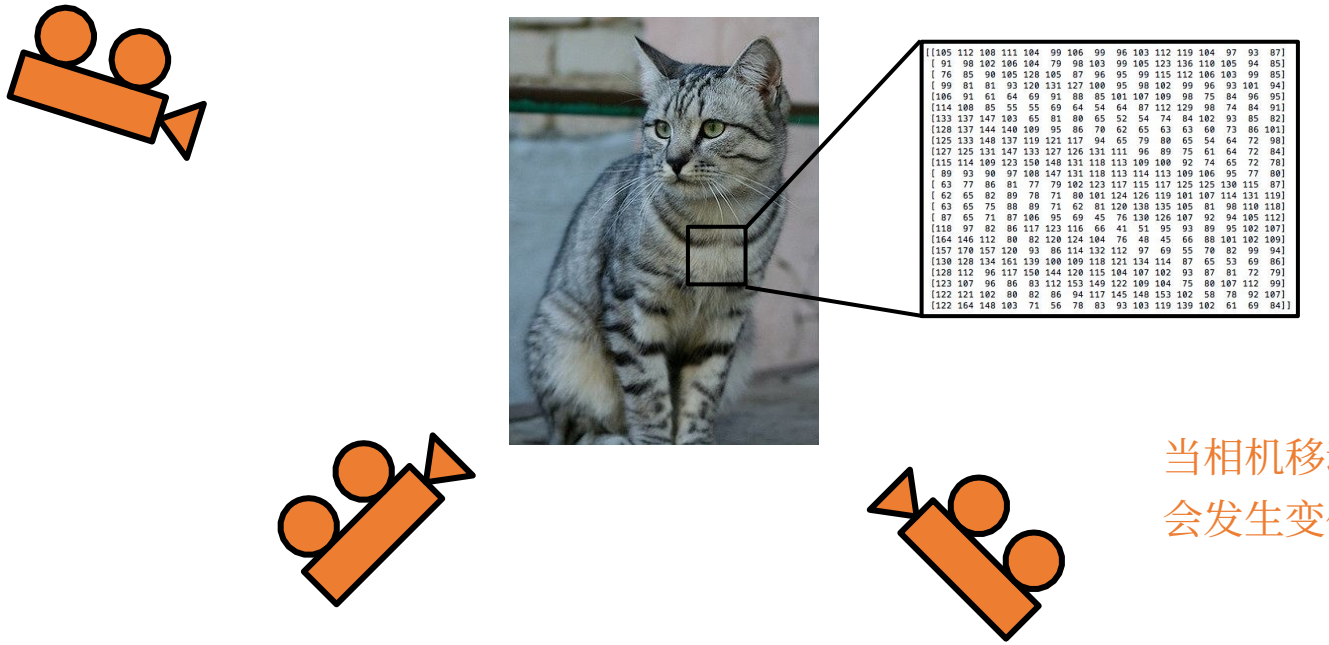
计算机看到的内容

图像是一个由介于 [0, 255]之间的整数组成的张量：

例如 800 x 600 x 3（3 通道 RGB）



挑战



当相机移动时，所有像素都会发生变化！

挑战：背景杂乱





挑战：光照



挑战：遮挡



挑战：变形





挑战：类内差异



我们如何对图像进行分类?

```
def classify_image(image):
  # Some magic here?
  return class_label
```

不同于例如对数字列表进行排序,

没有明显的方法来硬编码用于识别猫或其他类别的算法。

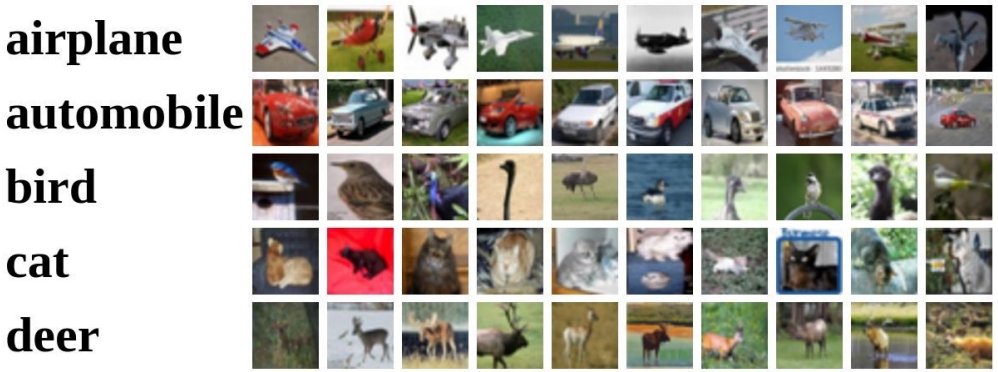


我们如何对图像进行分类?

1. 收集图像和标签的数据集 2. 提取图像的判别性特征 3. 使用机器学习算法训练分类器 4. 在新图像上评估分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```





最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```

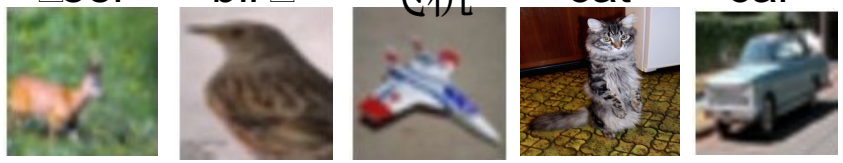
记忆所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

预测最相似训练图像的标签



最近邻



带标签的训练数据

?



查询图像

距离度量

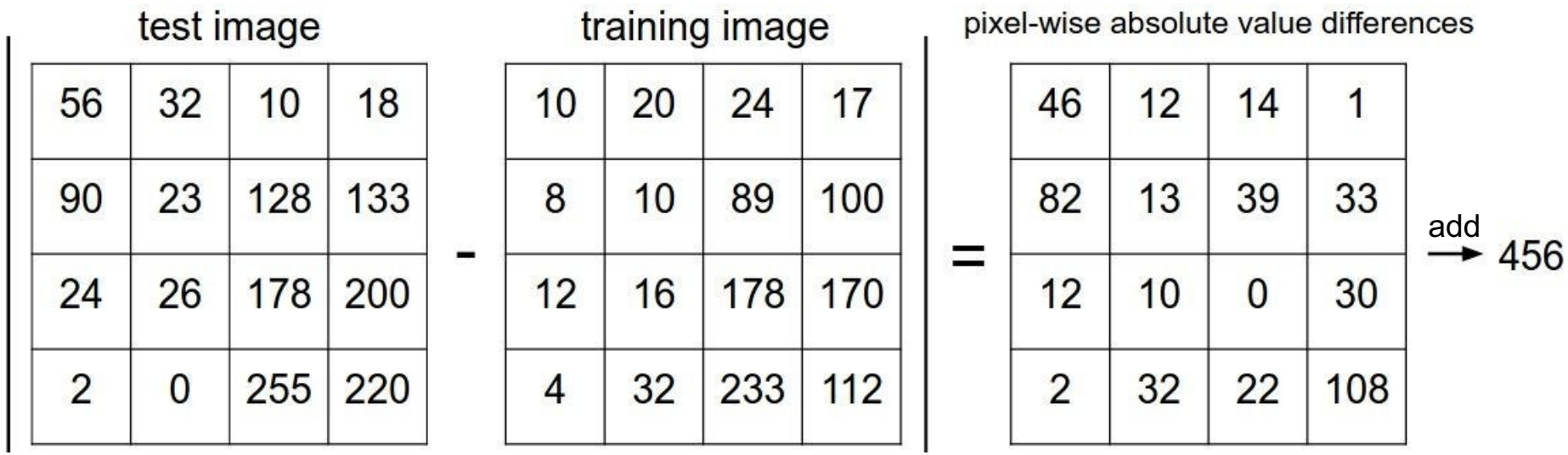


$$| \quad , \quad | \rightarrow R$$

Distance Metric 用于比较图像

L1 距离：

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

记忆训练数据

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

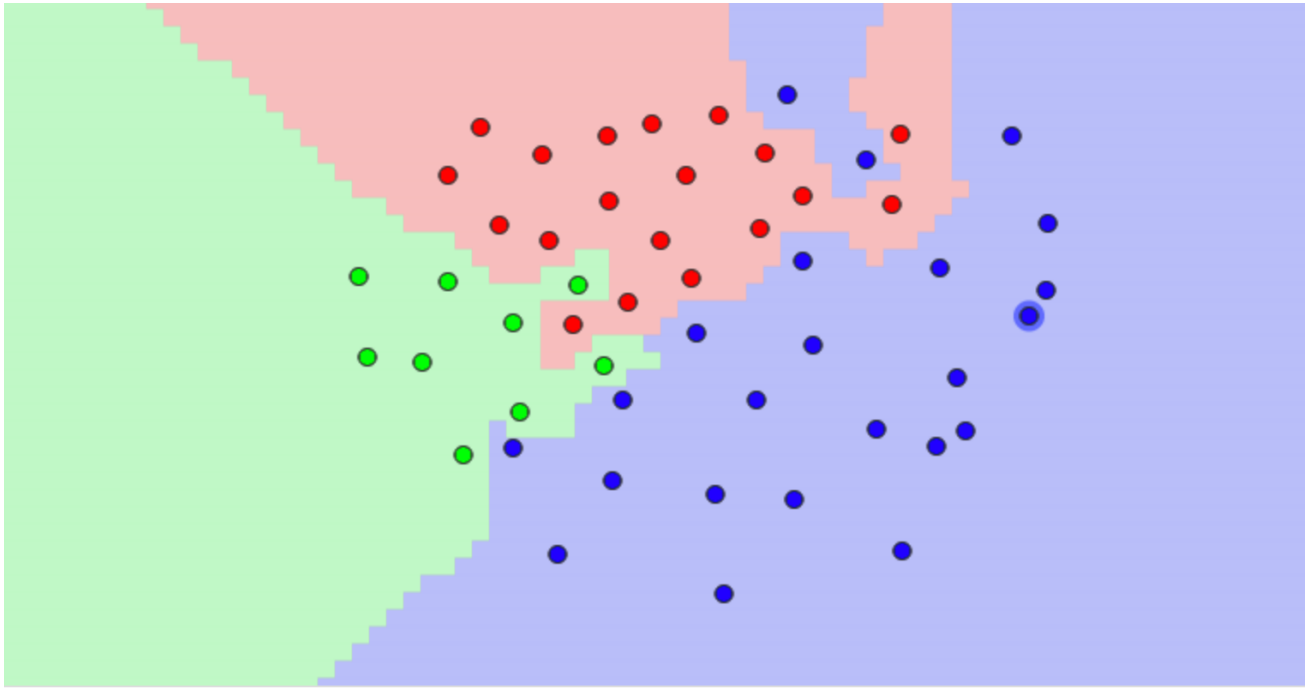
    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

对于每张测试图像：找到最接近的训练图像，预测最近图像的标签

这看起来像什么?

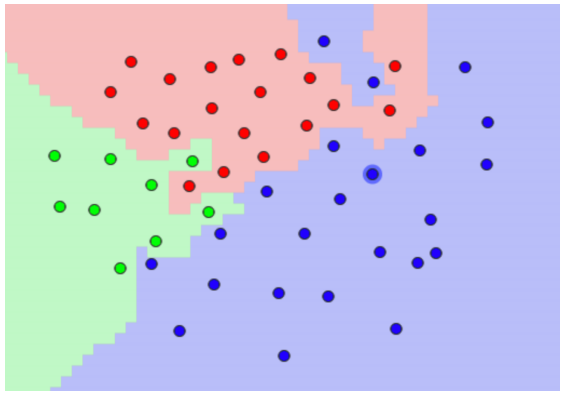


1-最近邻

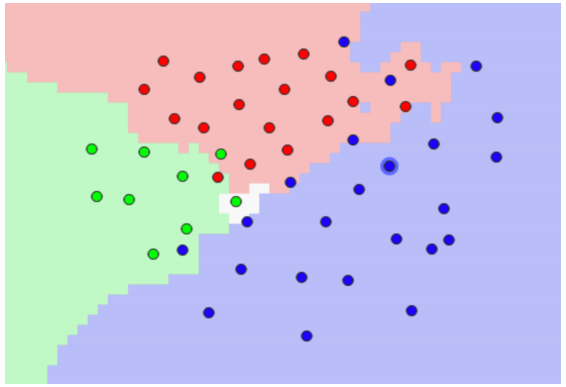


从最近邻到K近邻

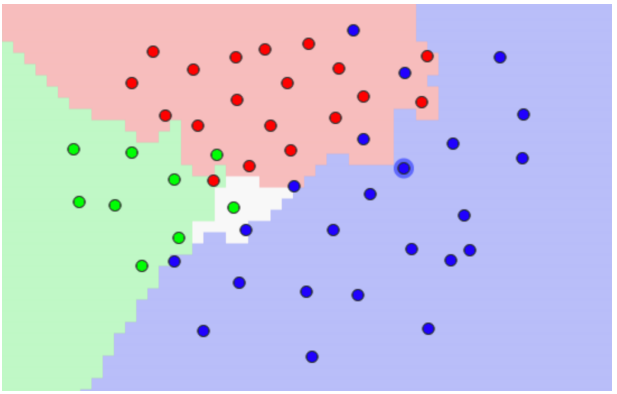
不从最近邻复制标签，而是从 K 个最近点中取**多数投票**



K r =



K r]



K r 5

一个交互式演示：https://lecture-demo.ira.uka.de/knn-demo/presetc=



超参数很重要!

要使用的最佳 □ 值是多少？要使用的最佳 **距离** 是什么?

这些是**超参数**：关于算法本身的选择。

非常依赖于具体问题和数据集。
必须把它们全部尝试一遍，看看哪种效果最好。



设置超参数

想法 #1：选择在 **训练数据** 上效果最好的超参数

□□□：K r = 在训练数据上总是表现完美





设置超参数

想法 #1：选择在**训练数据**上表现最佳的超参数

□□□：K r = 总是能在训练数据上完美运行



思路#2：选择在**测试**数据上表现最佳的超参数

□□□：不知道算法如何在新数据上的表现





设置超参数

想法 #1：选择在**训练数据**上表现最佳的超参数

糟糕：Kr = 在训练数据上总是表现完美



思路#2：选择在**测试**数据上表现最佳的超参数

糟糕：不知道算法如何将在新数据上表现



想法 #3：将数据划分为 **训练**集、**验证**集；在验证集上选择超参数，并在测试集上评估

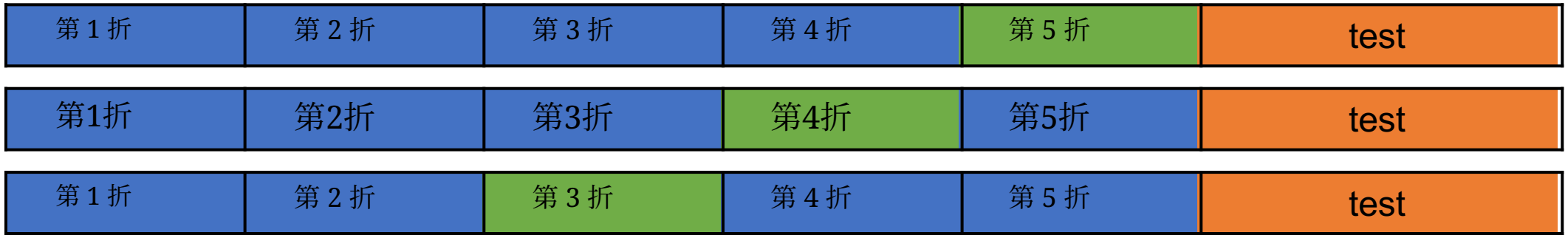
更好！





设置超参数

想法 #4：**交叉验证**：将数据分成多个 **折**，依次将每一折作为验证集，并对结果取平均



对小型数据集有用，但在深度学习中并不太常用

KNN 在 CIFAR10 中的结果

10 个类别，5000 张训练图像，10000 张测试图像



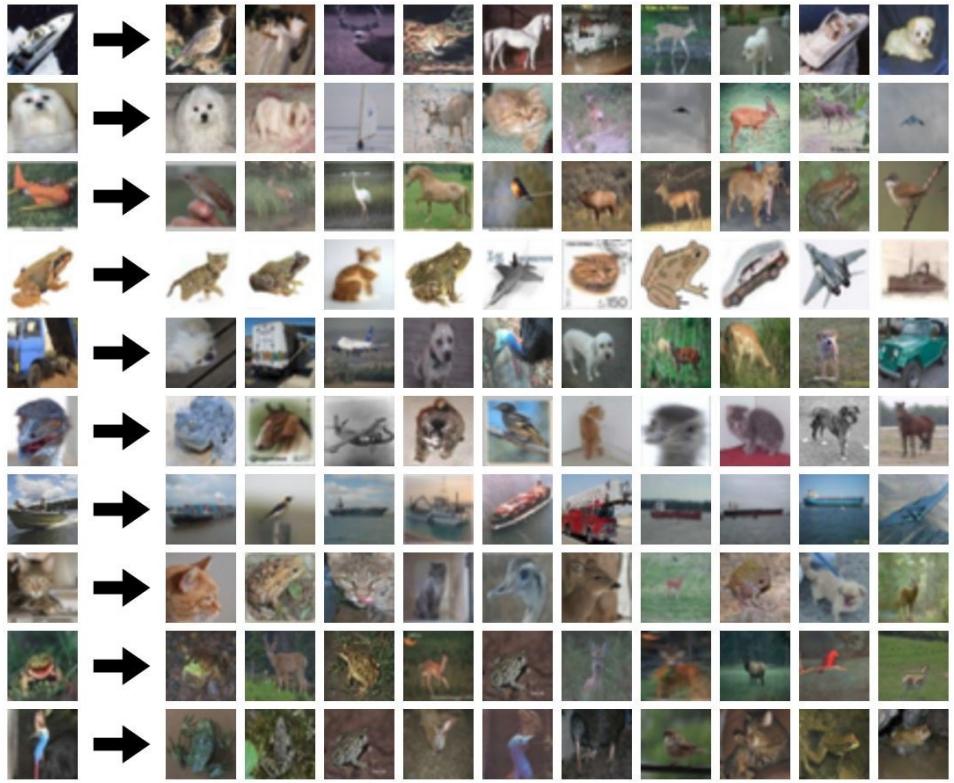


KNN 在 CIFAR10 中的结果

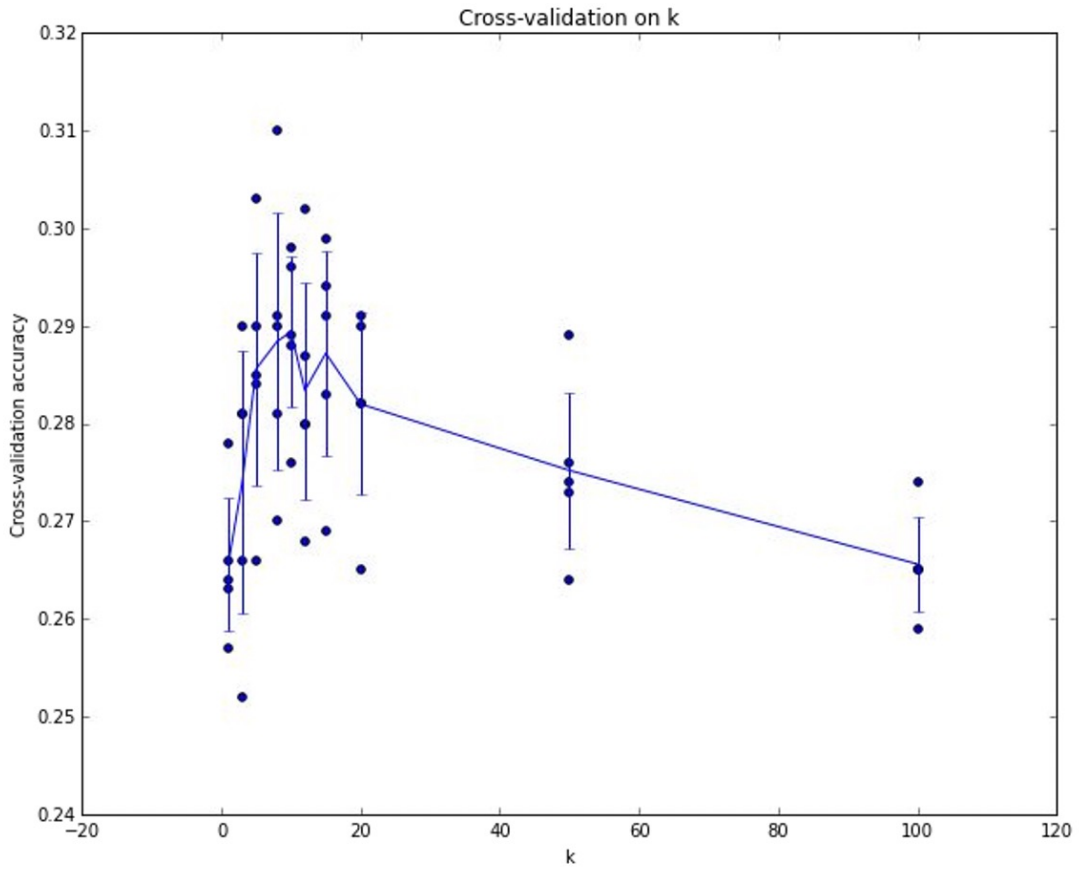
DB 个类别□B5BBB 张训练图像DB5BBB张测试图像



使用原始像素特征和最近邻的测试图像



设置超参数



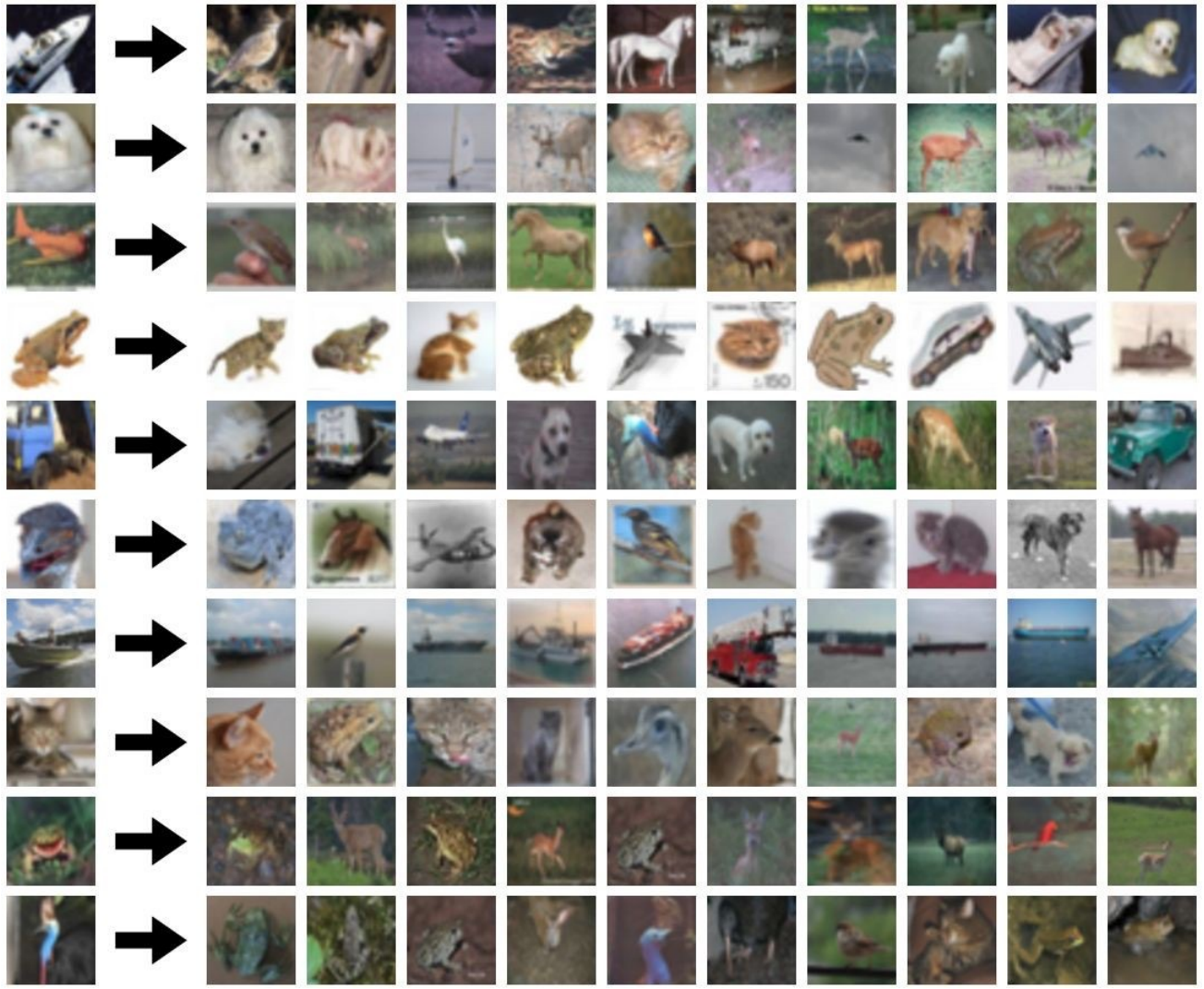
用于 **k.** 值的 5 折交叉验证示例

每个点：单个结果。

该线穿过均值，条形表示标准差。

（似乎 k □□ □ 对这些数据效果最好）

这看起来像什么?



这看起来像什么?



使用像素距离的 k 近邻**从未被使用**。

- 基于像素的距离度量信息量不足 - 测试时非常慢

需要进行特征提取！



原文



遮挡



打乱（1 像素）



重新着色

我们如何对图像进行分类?



特征提取

机器学习

我们如何对图像进行分类?



特征提取

机器学习

原始像素 方向梯度直方图（HoG） [1]局部二值模式（LBP） [2]尺度不变特征变换（SIFT） [3]

[1]“用于人体检测的方向梯度直方图。” CVPR 2015[] “用于纹理分类的局部二值模式算子的完整建模”, TIP2010[]“ ”SIFT：预测影响蛋白质功能的氨基酸变化。“ Nucleic acids research, 2013

我们如何对图像进行分类?



特征提取

机器学习

最近邻、线性分类器、
支持向量机.....

简要总结

- 在**图像分类** 中，我们从一个由图像和标签组成的**训练集** 开始，并且必须在**测试集**上预测标签
- **K-最近邻** 分类器根据 K 个最近的训练样本预测标签

- 距离度量和 K 是**超参数**

使用**验证集**选择超参数；

只在最后阶段在测试集上运行一次！



线性分类器



一种更典型的框架

K 最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```



记住所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



预测最相似训练图像的标签



更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义模型 $f_W(x)$ 定义损失函数 L $f_W(x), y$ 求解使 L 最小化的 W^*

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



根据 $f_{W^*}(x)$ 预测标签



一种更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义模型 $f_W(x)$定义损失函数

L $f_W(x)$, y求解 W^* 以最小化 L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$ 预测标签



问题形式化



$I \in R^{32\times32\times3}$



$x \in R^{3072\times1}$

向量化

$W(?, b, x)$

W, b

参数

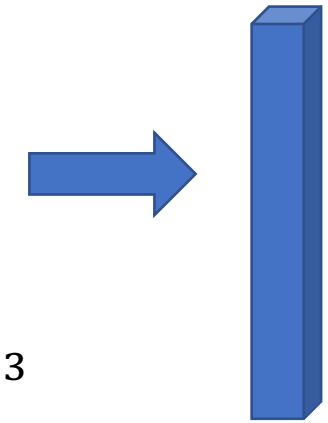
□□ 个数字给出类别得分

预测：最大得分

问题形式化



我 $\in R^{32\times32\times3}$



$x \in R^{3073\times1}$

向量化

$W(?, b)$

参数

□□ 个数字给出类别得分

预测：最大分数

通过增强该特征，我们可以消除偏差。



一个更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义模型 $f_W(x)$定义损失函数

L $f_W(x)$, y求解 W^*，使 L最小

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$ 预测标签



使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

给定一个由样本组成的数据集

$$\{x_i, y_i\}_{i=1}^{N}$$

其中 x_i 是图像，y_i 是标签。

$$L = \frac{1}{N}\sum_i L_i(f(x_i, \boldsymbol{W}), y_i)$$

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda r pysisp \end{cases}$$

其中 $\boldsymbol{s} = f\ x_i, \boldsymbol{W}\ \in R^3$ 是预测分数。

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

这意味着什么?

多类 SVM 损失:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$

其中 $\boldsymbol{s} = f\ x_i$、$\boldsymbol{W} \in R^3$ 是预测分数。

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \begin{matrix} j \neq y_i \\ 0 \end{matrix} \Big| \quad if\ s_{y_i} > s_j + 1 \quad \text{h}$$
$$s_j - s_{y_i} + 1 \quad o\lambda\ pysisp$$

对于除 targetone 之外的其他类别所对应的所有分数

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$ h

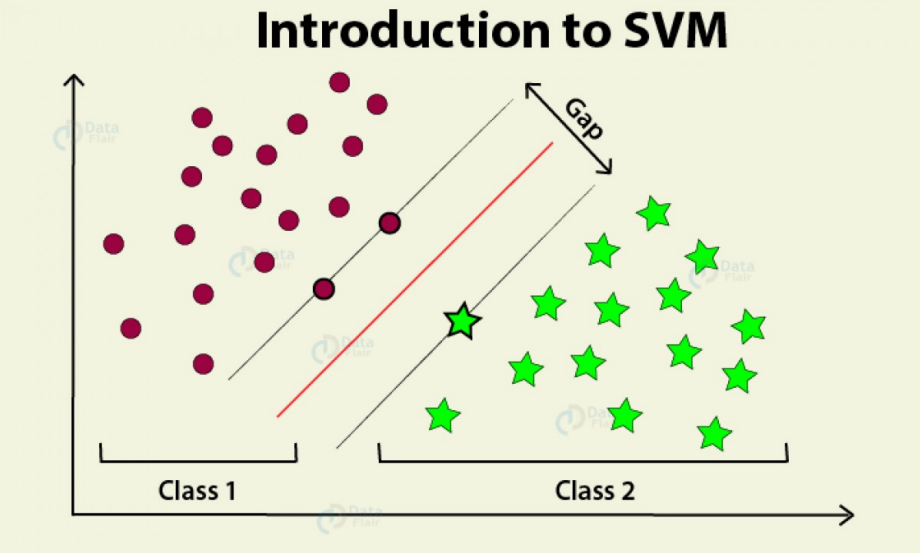
如果得分值小于目标类别的得分值加上一个间隔

3.7 n-1.a - 5

则损失为 0

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b

间隔：我们希望促使每两个类别之间由一个间隙分隔。



来源：https://morioh.com/p/74f4a4b78e0b

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda r pysisp \end{cases}$$

如果分数值小于目标类别的值加上一个 间隔

<.7 n 15.a - 5

那么损失为 0

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$

h

否则

3.7 1 >.5 - 5

损失是当前得分与目标得分之差再加上一个间隔

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

多类 SVM 损失：

$$L_i = \begin{cases} 0 & j \neq y_i \\ s_j - s_{y_i} + 1 & if\ s_{y_i} > s_j + 1 \\ & o\lambda\ pysisp \end{cases}$$ 小时

重写

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

利用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



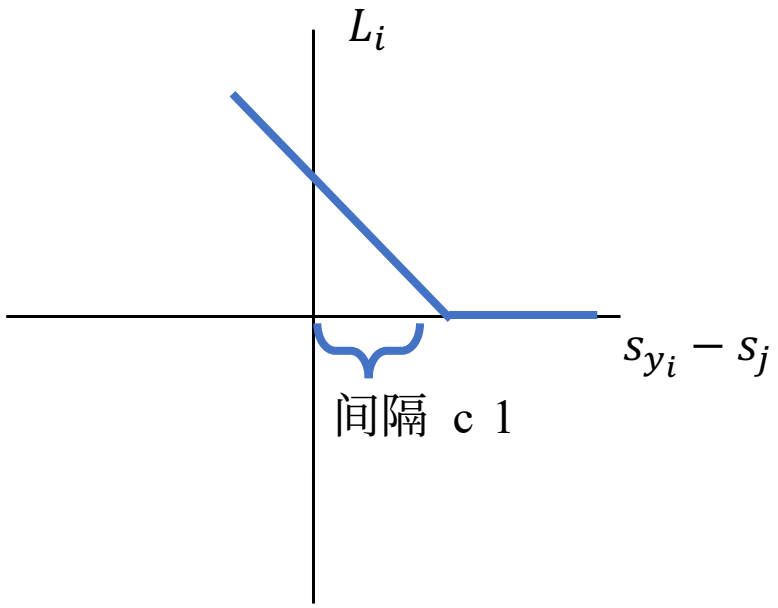


猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \mathrm{o\ g}, \left(0, s_j - s_{y_i} + 1\right)$$



使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9		

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

~ max(0, 5.1 - 3.2 3 1) 3 max(0, -1.7 - 3.2 3 1) ~ max(0, 2.9) 3 max(0, -3.9) ~ 2.t 3 L~ 2.9



使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9	0	

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

~ max(0, 1.3 - 4.t 3 1) 3 max(0, 2.0 - 4.t 3 1) ~ max(0, -2.6) 3 max(0, -1.9) ~ L 3 L~ L

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9	0	12.9

多类SVM损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

~ max(0, 2.2 - (-3.1) 3 1)3
max(0, 2.5 - (-3.1) 3 1)~
max(0, 6.3) 3 max(0,
6.6)~ r .f 3 r .6~ 12.9

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b





猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**损失

2.9 0 12.9

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \mathrm{o\ g}, \left(0, s_j - s_{y_i} + 1\right)$$

$$L = \frac{1}{e} \sum_{i=1}^{N} L_i$$

$L = (2.9 + \infty + -2.$

$9 \quad C)/.$

$= \leq \approx \sum$

示例代码

$$L_i = \sum_{j \neq y_i} \mathrm{o\ g}, \left(0, s_j - s_{y_i} + 1\right)$$

```
def L_i_vectorized(x, y, W):
  scores = W.dot(x)
  margins = np.maximum(0, scores - scores[y] + 1)
  margins[y] = 0
  loss_i = np.sum(margins)
  return loss_i
```



最优的W唯一吗?

$$f(x; W) = Wx$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\infty 1 y(x_i 25\ )_j \text{ } ?\ y(x_i 25\ )_{y_i} + \rightarrow\right)$$

假设我们找到了一个 W，使得 L□╞。这个 W 唯一吗?

不！nW 也会导致 L□╞。

我们如何在 W 和 nW 之间进行选择?



最优的W是否唯一?

$$f(x; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\infty 1[\mathbf{5}\ \boldsymbol{x}]_j \text{ } [\mathbf{5}\ \boldsymbol{x}]_{y_i} + \rightarrow\right)$$



最优的W是否唯一?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j \neq y_j} \text{o a0}\left(\infty 1[\boldsymbol{W}5\ \boldsymbol{x}]_j \text{ } [\boldsymbol{W}5\ \boldsymbol{x}]_{y_i} + \rightarrow\right)$$



最优的W是否唯一?

$$f(x;W)=Wx$$

$$L=\frac{W}{f}\sum_{i=1}^{N}\sum_{j\neq y_j}\text{o a0}\left(\infty 1[5\ x]_j\ ?\ [5\ x]_{y_i}+\vec{W}\right)$$

随着 n 增加，间隔会变小，这可能导致过拟合。

$$f(x;W)=Wx$$

正则化

$$N \leq \frac{\sum_{i=1}^{N} \square \quad N_i(+_i;\ \boldsymbol{x}), \lambda_i)}{R}$$

数据损失：模型预测应与训练数据匹配



正则化

$$N \leq \frac{\sum_{i=1}^{N} \square \quad N_i \; y_i; \; \boldsymbol{x} \; , \lambda_i)}{R} \quad -\partial a(\boldsymbol{x} \;)$$

为什么这个术语有效?

数据损失：模型预测应与训练数据匹配

正则化：对模型的偏好。

示例：a2 正则化：$R \; W \; = \; W \; \& \; = \leftarrow_k W_k^2$

L_1 正则化：$R \; W \; = \; W \; (\; = \leftarrow_k W_k$

$| \quad |$

正则化

a2 正则化： $R\ W\ =\ W\ \&\ =\leftarrow_k W_k^2 L_1$

正则化： $R\ W\ =\ W\ (\ =\leftarrow_k W_k\ \ \ \ |\ \ |$

1. “最大间隔”原理
在 SV 中 M，目标是找到一个能够分隔各个类别的决策边界 .
无正则化：模型可能会过度努力地正确分类训练数据中的每一个噪声点，从而导致非常复杂的边界。

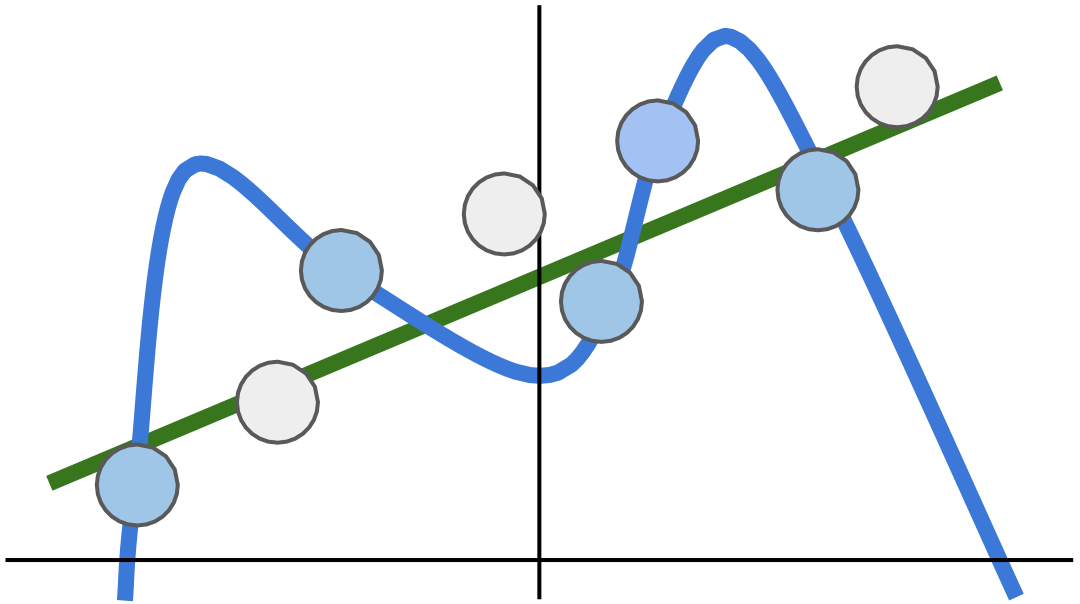
使用正则化：最小化 $W\ _2$ 在数学上等价于 **最大化间隔**。更大的间隔通常会在新的、未见过的数据上带来更好的性能，因为它更稳健。

2. 控制“权重”
为什么权重要小? 如果权重 W_i 非常大，那么特定输入特征 x_i 的微小变化也会导致巨大波动 g
在输出中。通过迫使权重变小，正则化确保模型不会过度依赖任何单一
特征 o 训练集中的 r 特定噪声。这会带来一个 **更平滑** 的模型。



正则化

如果两个模型对数据的解释效果相同，那么通常更简单的那一个才是正确的!



Occam's Razar：在多个相互竞争的假设中，最简单的就是最好的，William of Ockham 1285-1347



其他更复杂的正则化方法

- Dropout
  - Srivastava, Nitish, 等。“Dropout：一种防止神经网络过拟合的简单方法。” JMLR, 2014
- 批归一化
  - Ioffe, Sergey, 和 Christian Szegedy。“批量归一化：通过减少内部协变量偏移加速深度网络训练。” ICML, 2015.

Mixup

- Zhang, Hongyi, 等. “mixup：超越经验风险最小化。” □R □□2018.

□



SVM 分类器的分数不具备概率可解释性

0

解决方案：Softmax 分类器！



将预测得分转换为概率



概率必须为 □□ □

概率之和必须为 1

cat	**3.2**	**24.5**	**0.13**
car	5.1	164.0	0.87
frog	-1.7	0.18	0.00

ex□

归一化

Softmax 函数

$$\boldsymbol{a} = y(x_i, m)$$

$$x^s$$

$$\frac{x^{s_k}}{\leftarrow_j x_{,j}}$$

损失函数

比较两个概率分布：交叉熵

$$L = -\frac{1}{e}\sum_i u_i \ln(x_i) = -\frac{1}{e}\sum_i \ln\left(\frac{p^{s_{y_i}}}{\leftarrow_j p^{s_j}}\right)$$



概率必须为□□□

概率之和必须为1

cat	**3.2**	ex□	**24.5**	归一化	**0.13**	**1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

比较

$$\boldsymbol{a} = y(x_i, m)$$

$$x^s$$

$$\frac{x^{s_k}}{\leftarrow_j x_{,j}}$$

损失函数

比较两个概率分布：
交叉熵

$$L = -\frac{1}{e}\sum_i u_i \ln(x_i) = -\frac{1}{e}\sum_i \ln\left(\frac{p^{s_{y_i}}}{\leftarrow_j p^{s_j}}\right)$$

问题1：交叉熵损失可能的最小/最大值是多少？
答：当 $\boldsymbol{a}_{y_i}$ W N 时，即所有预测都正确，那么

$$L \; ? \; ? \; \frac{\rightarrow}{f}\sum_i \text{mg}(\rightarrow) = \infty$$

当 $\boldsymbol{a}_{y_i}$ W ∞时，即所有预测都错误，那么

$$L \; ? \; ? \; \frac{\rightarrow}{f}\sum_i \log\infty \; W \; N$$

损失函数

比较两个概率分布：
交叉熵

$$L = -\frac{1}{e}\sum_i u_i \ln(x_i) = -\frac{1}{e}\sum_i \ln\left(\frac{p^{s_{y_i}}}{\leftarrow_j p^{s_j}}\right)$$

问题2：在初始化时，所有 $\boldsymbol{a}$ 都将近似相等；损失是多少？
答：假设有 C 个类别，

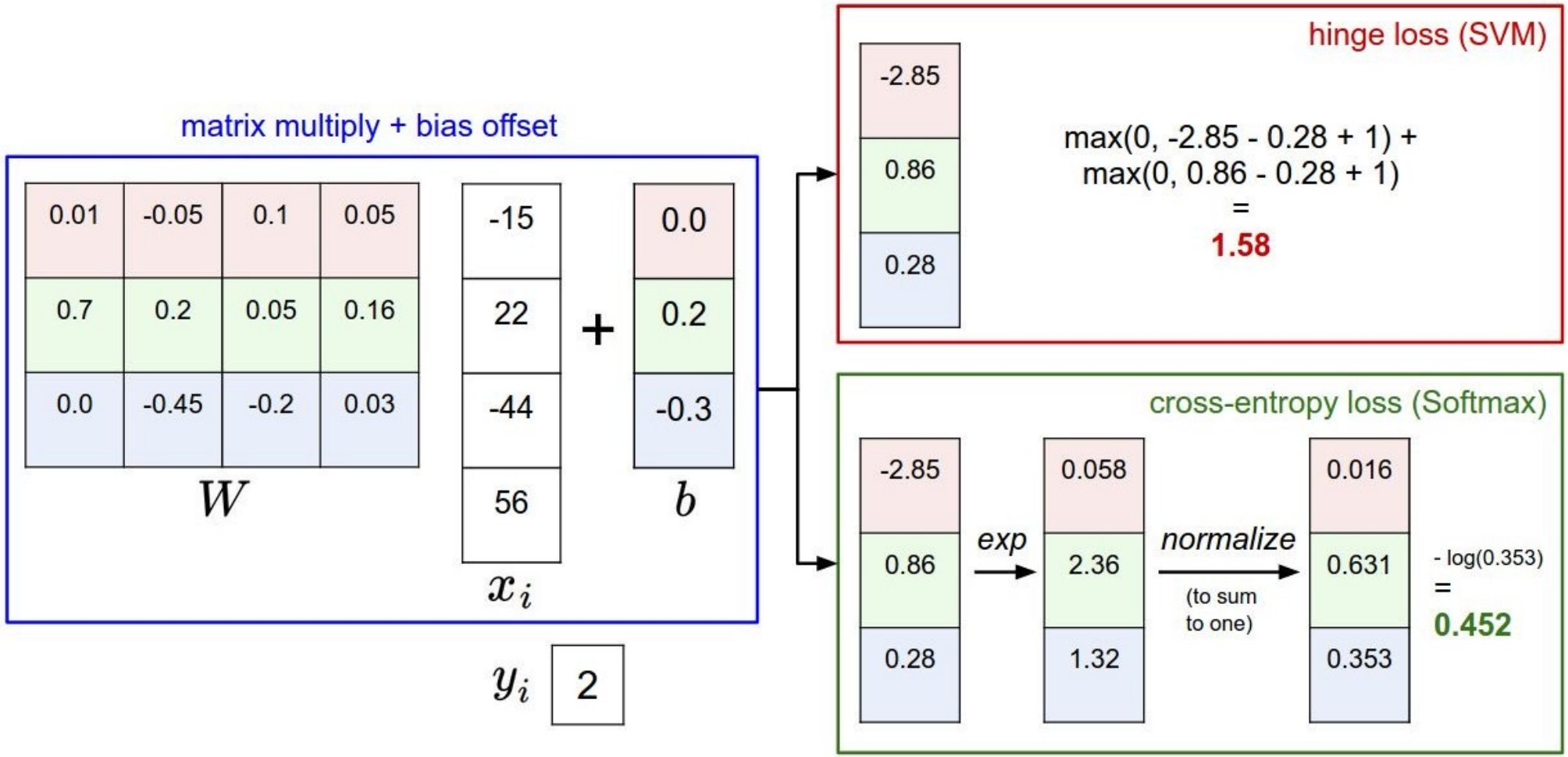
$$L = \vec{f}\sum_i \ln\left(\vec{b}\right)$$

如果 Cg_M，那么 $L = \log 1($ $)$ ≈ 2.3.

$a = y(x_i 1m)$ x^s $\frac{x^{s_k}}{\leftarrow_j x_j}$

在调试模型时，请牢记这些值。

比较 Softmax 分类器与 SVM



更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义一个模型 $f_W(x)$定义损失函数

L $f_W(x)$, y求解 W^* 从而最小化 L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$ 预测标签



随机搜索?

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
  W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
  loss = L(X_train, Y_train, W) # get the loss over the entire training set
  if loss < bestloss: # keep track of the best solution
    bestloss = loss
    bestW = W
  print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (trunctated: continues for 1000 lines)
```



随机搜索?

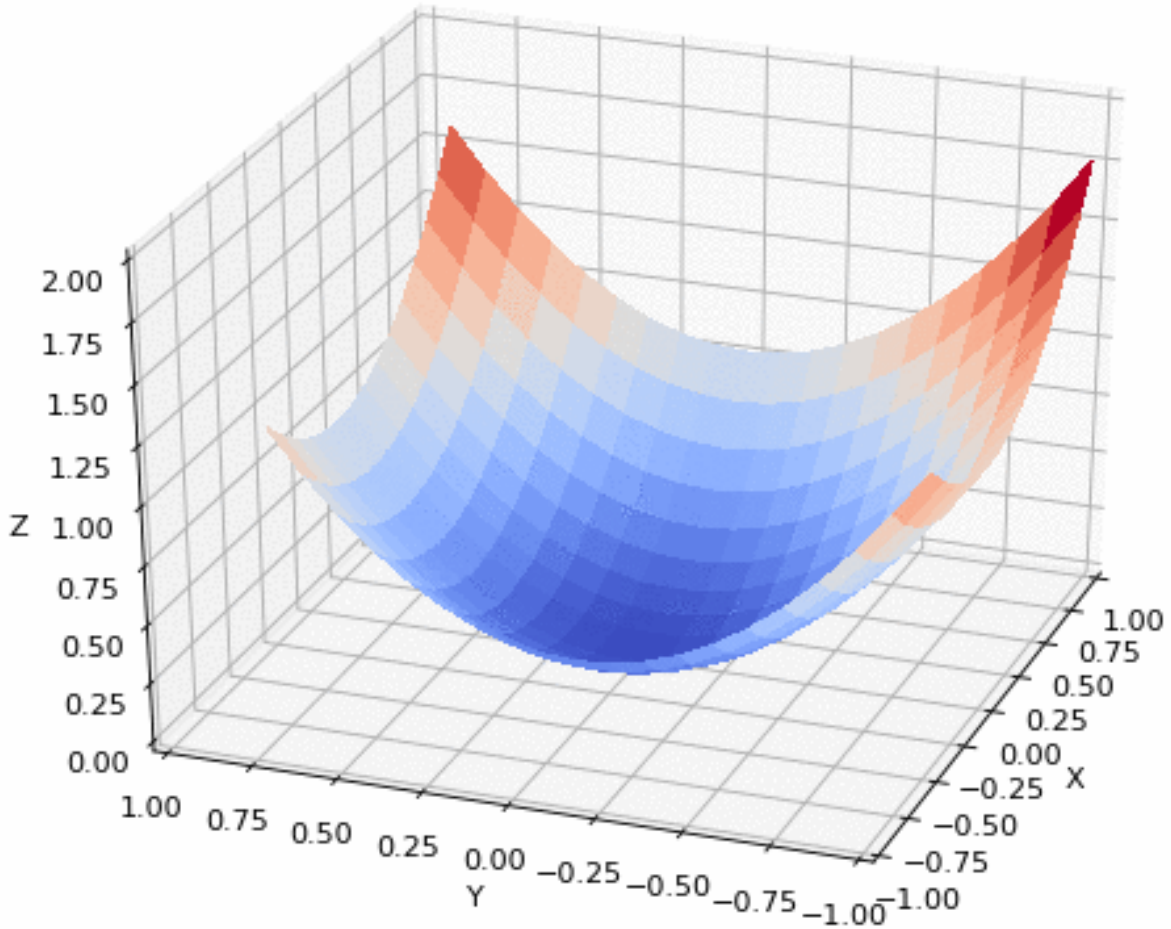
```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% 准确率（SOTA 为 □t t .7%）

随机搜索非常慢



梯度下降



思路：沿着使损失下降最陡的方向前进。

$$W \in W - -\frac{=L}{=W}$$

步长　　梯度

梯度下降

$$\boldsymbol{x} \cdot \boldsymbol{x} = \partial \frac{\mathbf{W}\mathbf{N}}{\mathbf{W}\mathbf{x}} \leq \boldsymbol{x} = \partial \frac{\mathbf{W}\frac{\sum}{R} \leftarrow_{i=1}^{N} N_i}{\mathbf{W}\mathbf{x}} \leq \boldsymbol{x} = \partial \frac{\sum}{R} \square_{i=1}^{N} \frac{\mathbf{W}N_i}{\mathbf{W}\mathbf{x}}$$

$$N_i \leq \square_{j \neq y_i} \max\left(1, \boldsymbol{x}_j^T y_i = \boldsymbol{x}_{y_i}^T y_i - \sum\right)$$



梯度下降

$$N_i \leq \sum_{j \neq y_i} \max\left(1, \boldsymbol{x}_j^T y_i = \boldsymbol{x}_{y_i}^T y_i - \sum\right)$$

当 $> {}_j^T x_i - > {}_{y_i}{}^T x_i + 1 \geq 0$ 时，则 $L_i = 0$，且梯度为 0

当 $> {}_j^T x_i - > {}_{y_i}{}^T x_i + 1 > 0$ 时，则

$$\frac{\boldsymbol{x} L_i}{\boldsymbol{x} \mathbf{5}_{y_i}} = \sum x_i \qquad \frac{\boldsymbol{x} L_i}{\boldsymbol{x} \mathbf{5}_j} = x_i$$



损失函数

- 我们必须定义一种误差度量。
- 我们的模型有多不准?

- 均方误差
  - 对于给定的 x，求实际 y 值与预测 y 值（$y = mx + c$）之差。
  - 对该差值求平方。
  - 求 X 中每个值的平方的均值。

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$



线性回归

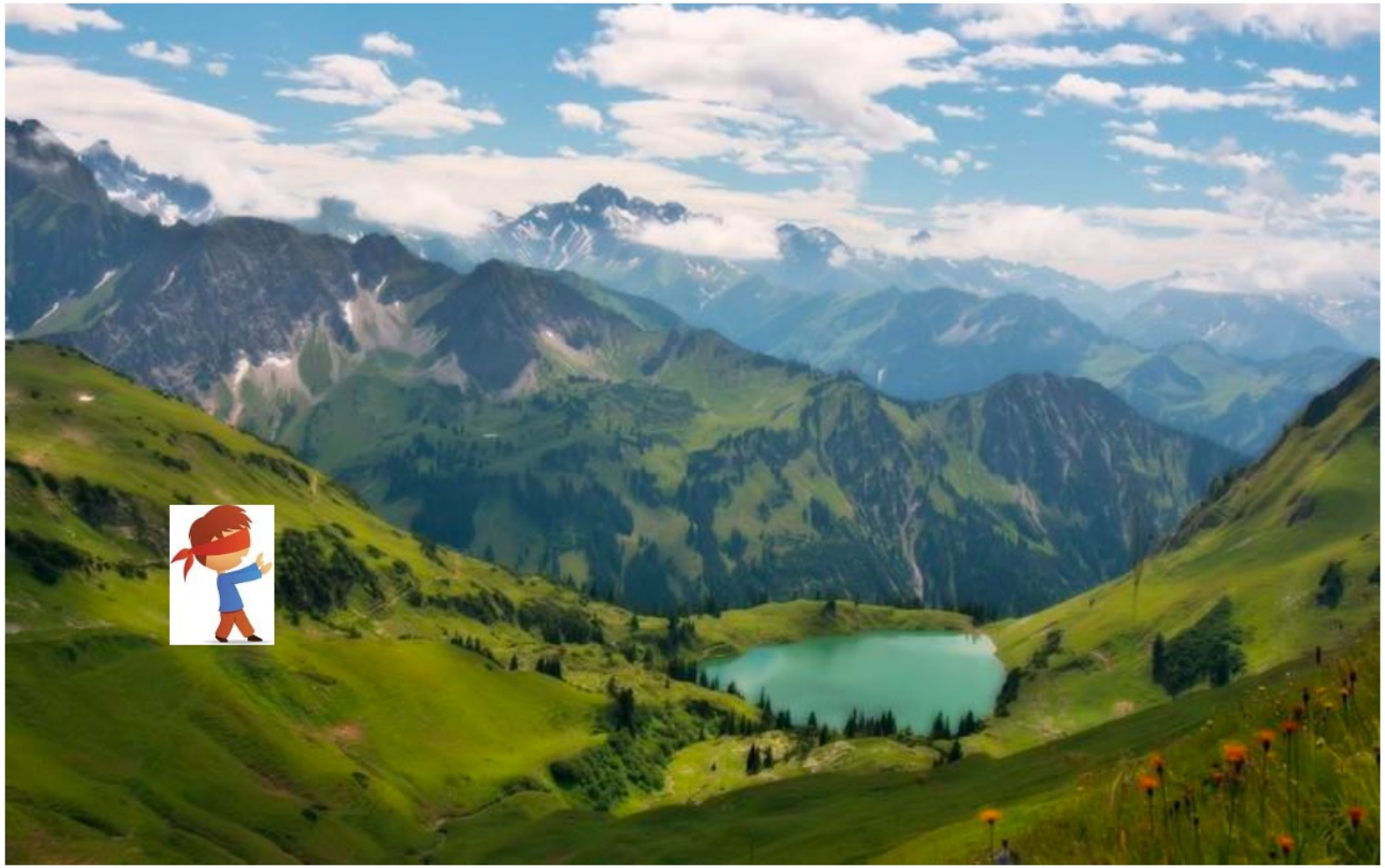
- 监督学习的一项核心原则

  寻找能够在所有可能数据上产生最小误差的模型参数。

- 我们只能观测到有限数量的数据，因此通常将其视为最小化训练数据上的误差（经验误差）。



优化

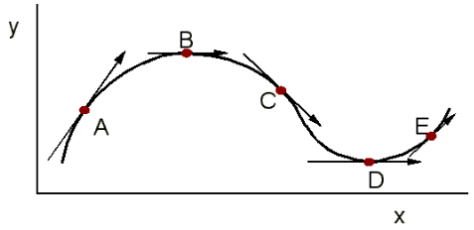


什么是梯度?

- 从几何上看
  - 指向函数增长最快的方向，其大小是该方向上图像的斜率

- 更形式化地说，在一维情况下

$$\frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



- 在高维情况下

$$\frac{\partial f}{\partial x_i}(a_1, \ldots, a_n) = \lim_{h \to 0} \frac{f(a_1, \ldots, a_i + h, \ldots, a_n) - f(a_1, \ldots, a_i, \ldots, a_n)}{h}$$



数值梯度

- 可以将 h 设为一个非常小的数值并计算：

$$\frac{\partial f(x)}{\partial x} = \frac{f(x+h)-f(x)}{h}$$

- 速度慢，且仅是一种近似
  - 需要对每个参数计算一次得分
  - 对 h 的选择敏感

- h 也需要选择——超参数



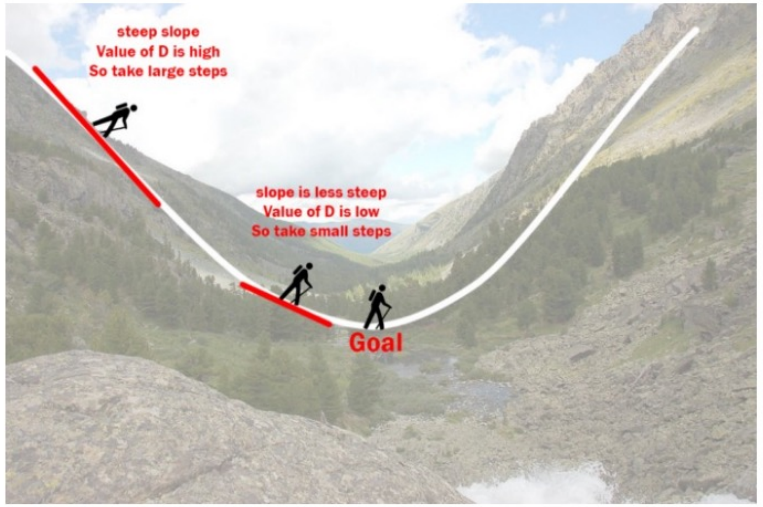
梯度下降

1. 初始时令 $\boldsymbol{D} = \infty$ 和 $c = \infty$ 令 L 为我们的学习率。它控制 $\boldsymbol{D}$ 的值在每一步变化多少。L 可以是一个较小的值，如 0.001，以获得良好的准确率。

2. 计算损失函数关于 $\boldsymbol{D}$ 的偏导数，并将 x、$\boldsymbol{c}$、$\boldsymbol{D}$ 和 c 的当前值代入其中，以获得导数值 $\boldsymbol{2}$。

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$



梯度下降



3. m 是关于 x 的偏导数值。类似地，我们来求关于 m、c 的偏导数：

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

4. 现在我们使用以下方程更新 m 和 c 的当前值：

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

5. 我们重复这一过程，直到损失函数的值非常小，或者理想情况下为 0（这意味着 0 误差或 100% 准确率）。此时剩下的 m 和 c 的值将是最优值。



迭代梯度下降

- 执行前向传播以计算损失
- 反向运行以计算梯度
- 对每个参数向量求损失和梯度的平均值
- 在每次迭代中，沿与梯度相反的方向更新每个参数向量，以使损失降低



梯度下降

更通用的解决方案?

自动微分!



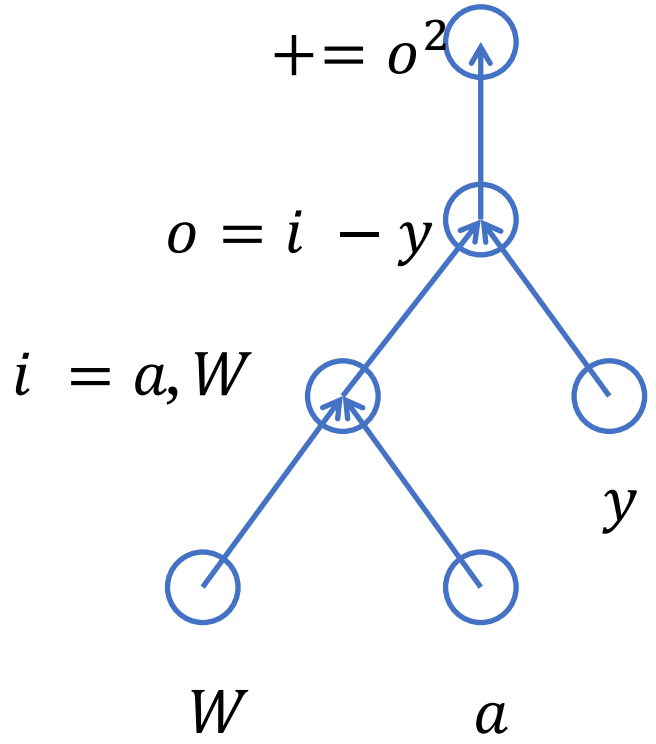
计算图

前向
- 分解为基本操作
- 构建有向无环图来表示计算

反向
- 通过链式法则求梯度

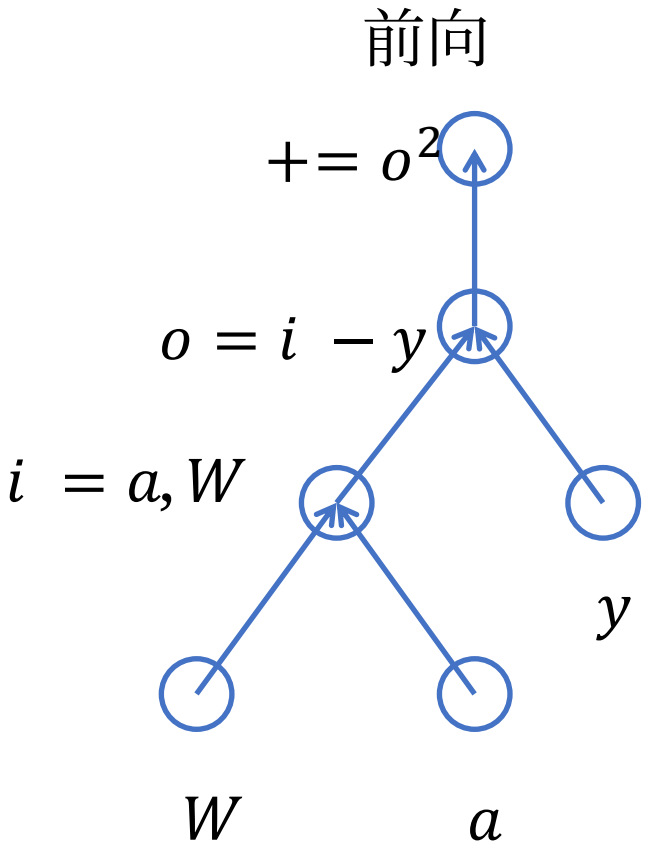
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial \partial_n} \frac{\partial \partial_n}{\partial \partial_{n-1}} 0 \frac{\partial \partial_2}{\partial \partial_1} \frac{\partial \partial_1}{\partial x}$$

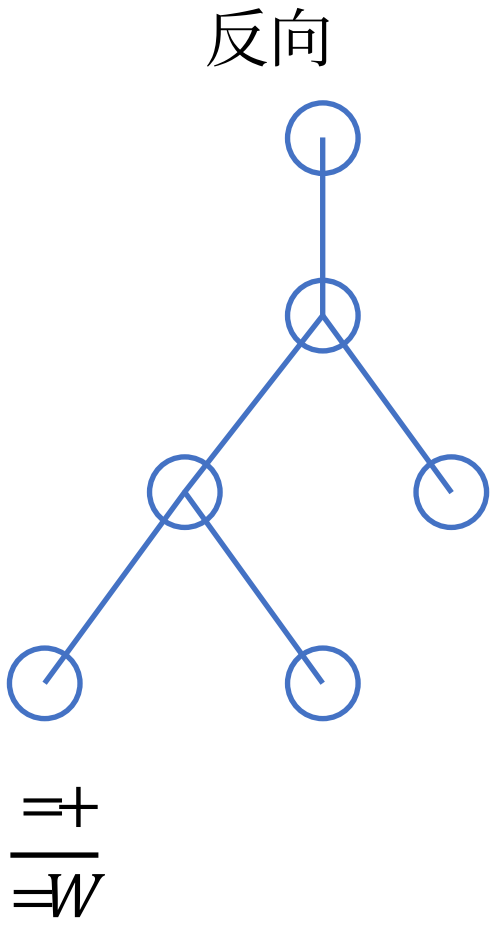
假设 $z = \mathbf{x}, \mathbf{w} - y\ 2$



反向累积

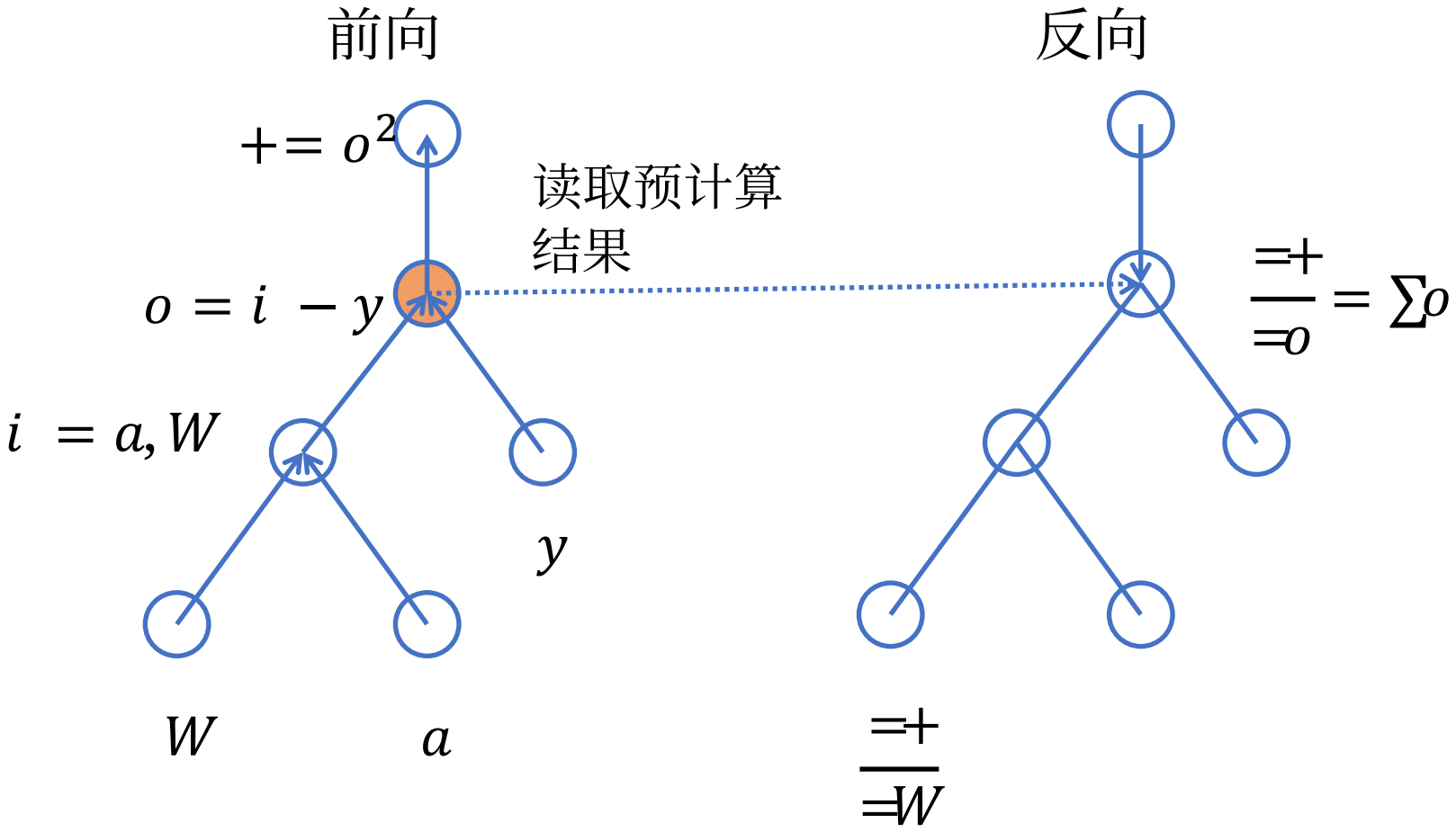
$z = \langle \mathbf{x}, \mathbf{w} \rangle - y$ 2 假设





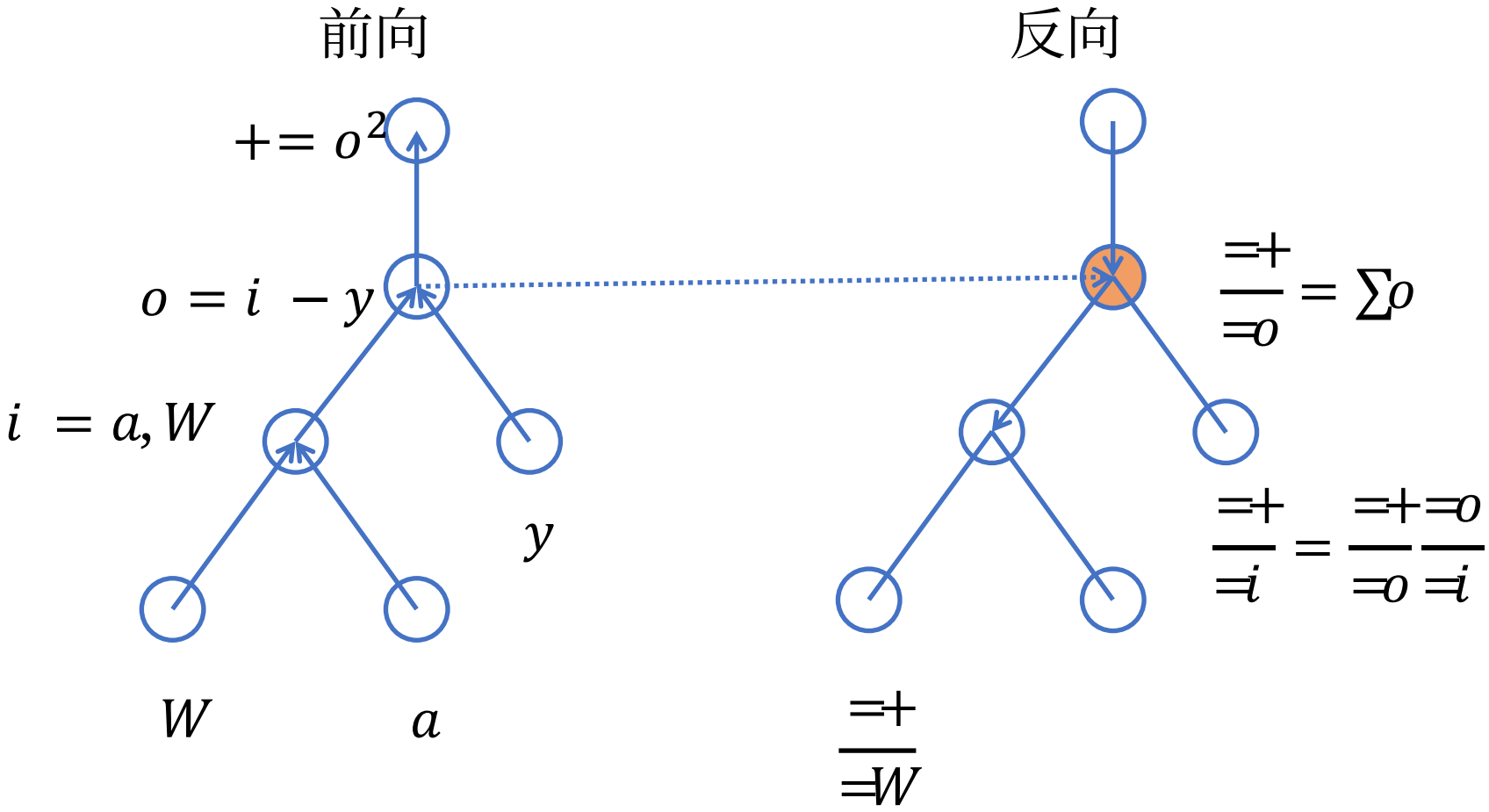
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y$ 2 假设



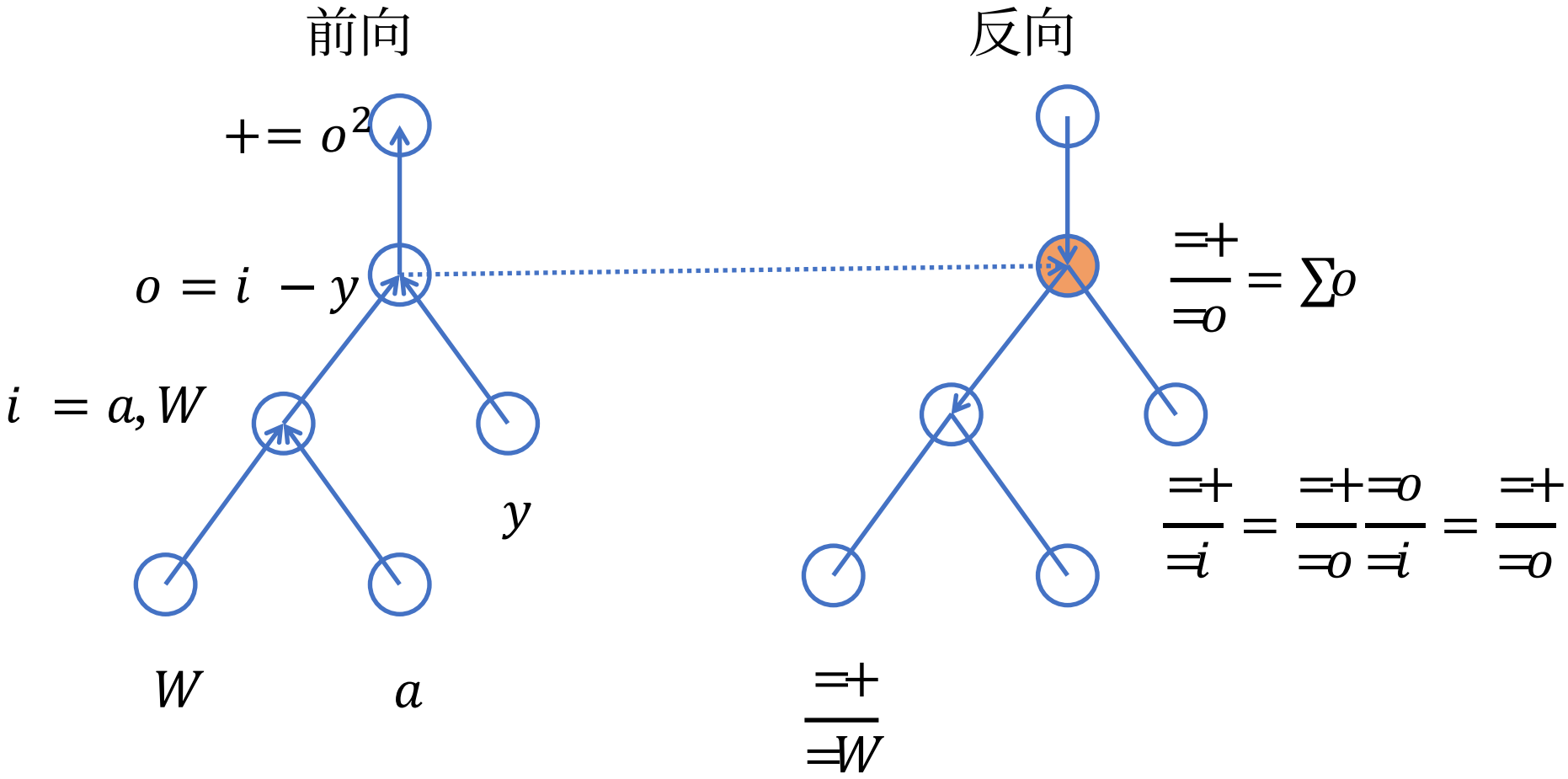
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



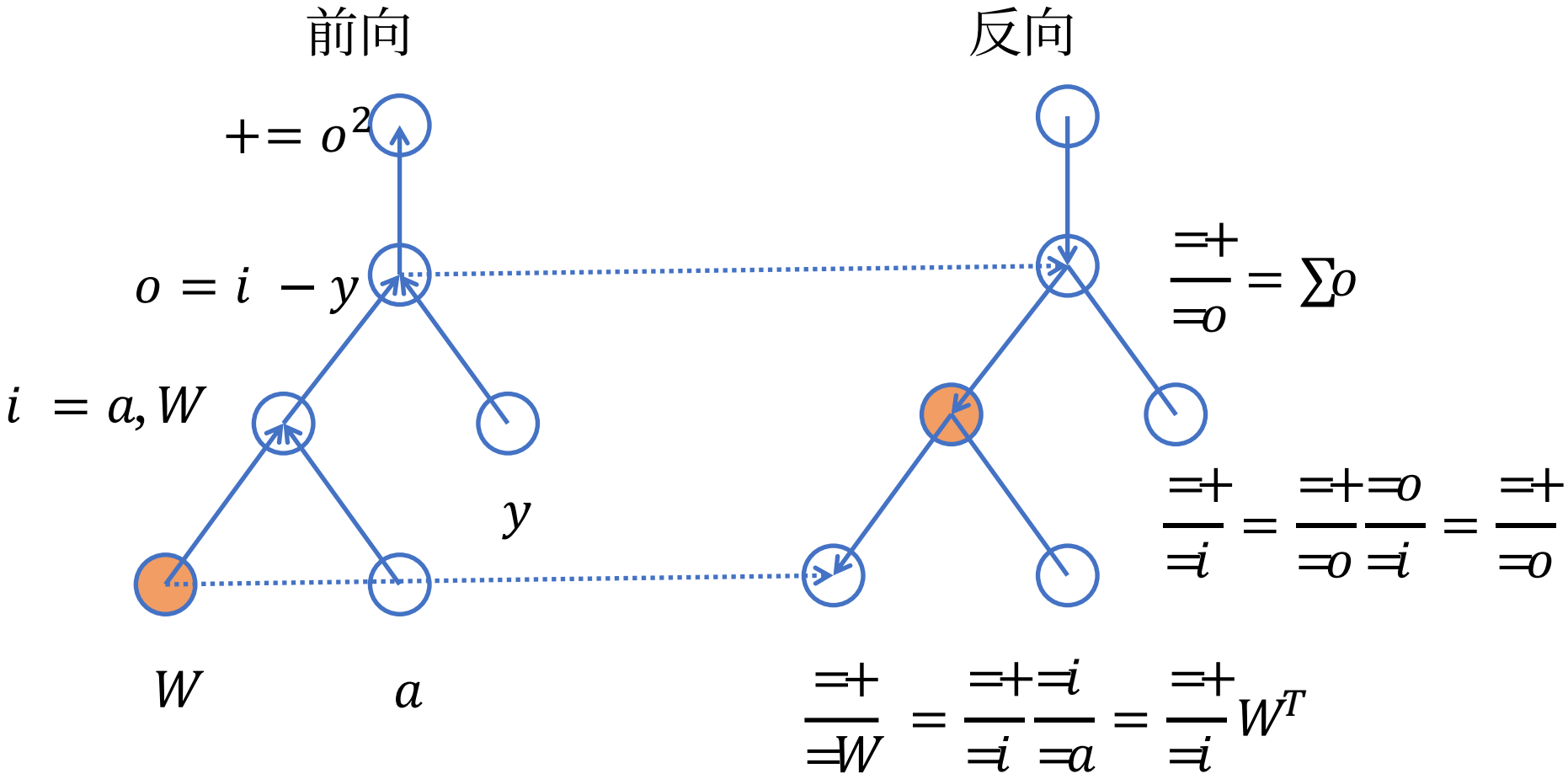
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



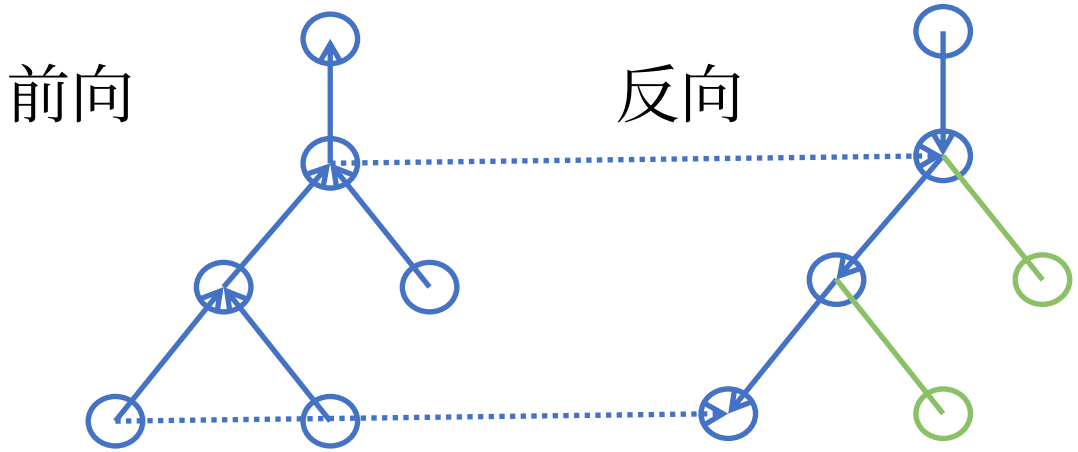
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y$ 2 假设



反向累积总结

- 构建计算图
- 前向：求值计算图，存储中间结果
- 反向：按相反顺序求值计算图
  - 消除不需要的路径





随机梯度下降

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial W}$$

当 N 很大时计算代价高昂

使用一批样本进行近似。B 可以根据内存/学习速度来选择……

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{B} \sum_{i=1}^{B} L_i}{\partial W}$$

